

FinAccrual ERP Platform

Project and Module Structure

Folder layout, architecture and responsibilities of every module

Item	Detail
Project	FinAccrual ERP Platform (Backend API, Excel Add-in, Admin Dashboard)
Document	Project and Module Structure

This document explains how the FinAccrual repository is organised, what each application and module is responsible for, how a request travels through the layers, and where to find or add code.

1. Repository Overview

FinAccrual is a monorepo containing three applications that work together: a **Backend API**, a **Microsoft Excel task-pane Add-in** (the main user interface) and an **Admin Dashboard**.

ERP-main/

└─ Backend/

└─┬─ Backend/ Express 5 API (real backend)

└─ Frontend-React/ Excel Add-in (React 19, Office.js, Webpack)

└─ Admin/ Admin Dashboard (React 19, TypeScript, Vite)

└─ docker-compose.yml Nginx container (port 8085)

└─ nginx.conf Load-balancing config → backend 8000 / 8001

└─ README.md LICENSE .gitignore

└─ package-lock.json (root lock file, no root package.json)

└─ localhost.session.sql (empty editor session file)

Application	Role	Talks to	Key technologies
nBackend API	Business logic, authentication, ERP integration, validation, persistence	MySQL, QuickBooks Online, Xero, Google, Microsoft	Express 5, Sequelize, mysql2, JWT, Joi, axios, exceljs, helmet, pino
Excel Add-in	User interface inside Excel: sign-in, plans, connect company, pull master data into sheets	Backend API, Office.js (Excel API)	React 19, Office.js, Webpack 5, Babel
Admin Dashboard	Back-office web console (clients, schedules,	Backend API (login only at	React 19, TypeScript, Vite,

	journals, exports, workpapers)	present)	Redux Toolkit, React Router 7, Axios, sonner
--	--------------------------------	----------	--

Architecture style: the backend is a **modular monolith** — one deployable process, divided into isolated domain modules plus a shared core layer. The add-in and dashboard are single-page React applications that call the backend over HTTPS/JSON.

2. System Context

Excel (task pane) Admin web console

Frontend-React ————— Admin —————

Office.js writes sheets | | HTTPS / JSON (JWT Bearer)



—————

| Nginx :8085 (optional) |

—————



—————

| Backend API (Express) :8000 |

| core + auth | admin | billing |

| quickbooks | xero | notifications |

| excelValidation |

|_____|



MySQL QuickBooks / Google /

(Sequelize) Xero APIs Microsoft OAuth

3. Backend Structure (Backend/Backend)

3.1 Folder tree

Backend/Backend/

|— package.json package-lock.json .env.example LICENSE

|— README.md BACKEND_DOCUMENTATION.md

|— **db_stub.js** test stub for the database module

|— debug_output.html leftover debug capture (not used)

|— scripts/

| |— loadtest.js autocannon load test

|— tests/

| |— integration/ 4 route tests

	└─ unit/ 12 unit tests	
	└─ src/	
	└─ server.js	entry: HTTPS, DB sync, listen
	└─ app.js	Express app and global middleware chain
	└─ routes/index.js	mounts routers + /connections
	└─ config/logger.js	pino logger (used by sync services)
	└─ core/	shared infrastructure (section 3.3)
	└─ modules/	business domains (section 3.4)

3.2 Startup sequence

1. server.js sets IPv4-first DNS and, on macOS in non-production, tries to load the Office dev certificates; it sets BACKEND_HTTPS accordingly.
2. app.js is loaded: middleware chain and routes are built (section 3.5).
3. core/database creates the Sequelize connection and registers all models and associations.
4. sequelize.sync({ alter }) creates or updates tables.
5. startNotificationCleanupJob() removes notifications older than 24 hours (now and hourly).
6. The HTTP (or HTTPS) server listens on PORT (default 8000).

3.3 The core layer (src/core)

Shared code that every module may use. Modules must not depend on each other's internals; they communicate through core or through the event bus.

Path	Responsibility
config/index.js	Loads .env and exposes typed settings (PORT, DB, TRIAL, QB, XERO, GOOGLE, MICROSOFT, secrets)
constants/index.js	Provider URLs (QuickBooks, Xero), scopes and the success HTML pages for OAuth pop-ups
database/index.js	Sequelize instance, model registration and associations; exports sequelize, User, Admin, QuickBooksToken, XeroToken, Notification
errors/AppError.js	Base AppError and subclasses (SessionExpired, Validation, Authentication, Authorization, LimitReached, ConnectionRefused, ErpSessionExpired)
errors/asyncHandler.js	Wraps async handlers so rejected promises reach the error middleware
events.js	Global EventBus; used for user.downgraded so billing and ERP modules stay decoupled
helpers/index.js	OAuth state generator, Basic-auth encoder, "blocked" pop-up page renderer
interfaces/IConnectionService.js	Contract for ERP connection services used by billing
logger/index.js	Thin console wrapper (info , warn , error) used by most code
validation/schemas.js	All Joi request schemas (auth, plans, connections, pull-master-data, billing, notifications, Excel validation)
oauth/TokenManager.js	Provider-independent access-token check, refresh, lock and revoke handling
oauth/interfaces/*	IOAuthClient , IOAuthTokenRepository , ILockManager contracts

<code>oauth/locking/LocalLockManager.js</code>	In-process per-company lock that prevents concurrent token refreshes
<code>oauth/errors/OAuthErrors.js</code>	<code>OAuthTokenRevokedError</code> (401) and <code>OAuthTokenRefreshError</code> (502)
<code>redis.js</code>	Unused Redis client stub; refers to missing config and package (see section 9)

Core middleware (src/core/middleware)

File	Purpose
responseTime.js	Adds X-Response-Time header and warns on slow requests
validateHeaders.js	Requires application/json on POST / PUT / PATCH bodies
sanitize.js	Cleans strings in req.body and req.params
rateLimiters.js	generalLimiter, authLimiter, oauthLimiter
validate.js	validate(schema, source) — runs a Joi schema on body or query
authMiddleware.js	authenticateJWT for admin routes
authorize.js	authorize(...roles) role check
oauthMiddleware.js	Verifies the OAuth state for QuickBooks and Xero callbacks
ensureValidToken.js	Resolves a valid ERP access token onto req.oauth
errorHandler.js	notFoundHandler and the central errorHandler that formats every error

3.4 Domain modules (src/modules)

Modules follow a layered file pattern. Large modules add sub-folders (services/, oauth/, views/).

File	Layer	Responsibility
routes.js	HTTP routing	Maps URLs to controller methods and attaches middleware
controller.js	HTTP layer	Reads the request, calls the service, shapes the response. No business rules
service.js	Business logic	Orchestrates repositories and external API calls
repository.js	Data access	Only place that queries Sequelize models
mapper.js	Transformation	Converts raw API/DB data into clean DTOs
model.js	Schema	Sequelize model for the tables the module owns
index.js	Public entry	What other code is allowed to import from the module

auth — user sign-in and sessions

modules/auth/

- |— auth.routes.js /api/auth/* routes
- |— auth.controller.js HTTP handlers
- |— auth.service.js sign-in, tokens, refresh, trial
- |— auth.middleware.js authenticate (JWT + active-user check)
- |— auth.validation.js simple e-mail / password checks
- |— jwt.service.js sign and verify JWT; generate refresh token
- |— google.service.js Google OAuth HTTP calls
- |— microsoft.service.js Microsoft Entra ID OAuth + Graph profile
- |— user.model.js `users` table
- |— user.repository.js user queries
- |— views/oauthPopup.view.js HTML returned to the sign-in pop-up

admin — legacy administrator module

routes.js, controller.js, service.js, repository.js, mapper.js, model.js, index.js.

Provides /api/admin/login, /signup, /me, /users. The model maps to the admins table; the mapper removes the password hash before returning data.

billing — plans and checkout

billing.routes.js, billing.controller.js, billing.service.js, index.js. The service is pure domain logic (plan weights, downgrade detection) that receives the user repository by injection and emits user.downgraded. The controller also renders the mock secure-checkout HTML page used in the pop-up.

quickbooks and xero — ERP integrations

The two modules are deliberately symmetrical. A thin **facade** (service.js) keeps one stable API and delegates to specialised sub-services.

modules/quickbooks/ (modules/xero/ mirrors this layout)

- |— routes.js controller.js
- |— service.js facade over the sub-services
- |— repository.js mapper.js model.js
- |— oauth/
 - | |— QuickBooksOAuthClient.js
 - | |— QuickBooksTokenManager.js
 - | |— QuickBooksTokenRepository.js
- |— services/
 - |— qbApiClient.js queries, paging, retries
 - |— qbEntityService.js entity fetchers
 - |— qbSyncService.js paged master-data sync
 - |— qbConnectionService.js connection lifecycle

modules/xero/ (differences from QuickBooks)

- |— services/
 - | |— xeroApiClient.js request headers, calls
 - | |— xeroEntityService.js entity fetchers
 - | |— xeroAuthService.js code exchange, tenant selection

| — xeroConnectionService.js lifecycle + master-data sync

— views/companySelection.view.js organisation picker page

Sub-component	Responsibility
OAuth client	Calls the provider's token endpoint to refresh tokens (extends IOAuthClient)
Token repository	Reads and writes token rows, converts expiry values, marks a company Disconnected (extends IOAuthTokenRepository)
Token manager	Instance of the shared TokenManager wired with that provider's client, repository and lock
API client	Executes provider requests with a valid token (QuickBooks adds paging and back-off retries)
Entity service	Customers, vendors, accounts, classes, locations, company / organisation info
Sync / connection service	Connection list, activate, rename, disconnect, plan limits, record counts and the paged master-data pull with isNew / isUpdated flags and a resume cursor
Xero company-selection view	HTML page where the user chooses which Xero organisations to connect

notifications — per-user history

notification.model.js, controller.js, routes.js, index.js. Stores notifications per user and removes entries older than 24 hours. Routes: list, create, mark-read, clear.

excelValidation — workbook checks

modules/excelValidation/

- |— routes.js controller.js service.js index.js
- |— parser.js .xlsx buffer → plain JS (exceljs)
- |— schemas/masterDataSchemas.js expected columns and types
- | per sheet (7 schemas)
- |— validators/
 - | |— schemaValidator.js headers and structure
 - | |— dataTypeValidator.js per-cell type checks
 - | |— typeCheckers.js reusable type predicates
- |— compare/
 - | |— diffEngine.js generic record-set diff
 - | |— apiComparator.js workbook vs live QuickBooks / Xero data
 - | |— dbComparator.js workbook vs database rows
- |— report/reportGenerator.js PASS / FAIL report (JSON or .xlsx)

The workbook is sent as base64 text inside a JSON body (limit 15 MB) so the global application/json header rule can stay in force.

3.5 Global middleware order (app.js)

SLno	Middleware	Purpose
1	responseTime	Timing header and slow-request warning
2	helmet	Security headers
3	cors	Cross-origin access with credentials
4	validateContentType	JSON header check
5	express.json({ limit: "15mb" })	Body parsing
6	sanitizeInput	Clean string input
7	express-session	Server session cookie (24 h)
8	generalLimiter + /api router	Rate limit then all routes
9	notFoundHandler, errorHandler	Unknown routes and central error format

3.6 API route map

Prefix	Module	Main endpoints
/api/health	routes/index.js	Server and database status
/api/auth	auth	google / microsoft connect + callback, refresh, logout, me, update-plan, start-trial (signup / login disabled)
/api/google/*, /api/microsoft/*	routes/index.js	Backward-compatible aliases of the OAuth connect / callback routes
/api/admin	admin	login, signup, me, users
/api/subscription, /api/payments	billing	upgrade, checkout page, complete
/api/quickbooks	quickbooks	connect, callback, disconnect, tokens, customers, vendors, accounts, classes, locations, company, export, connections (list, stats, delete, activate, count, rename, record-count), pull-master-data, refresh-incremental
/api/xero	xero	Same set as QuickBooks plus select-companies, contacts, organisation
/api/connections	routes/index.js	Provider-neutral list, stats, delete, activate, count, rename across QuickBooks and Xero

/api/pull-master-data	routes/index.js	Provider-neutral master-data pull; JSON or Server-Sent Events stream (stream=true) with cursor and mode=incremental
/api/notifications	notifications	list, create, mark-read, clear
/api/excel-validation	excelValidation	schema-check, data-type-check, report, vs-api, vs-database

3.7 Layered request flow

HTTP request

- global middleware (section 3.5)
- routes.js (authenticate, validate(Joi), limiter)
- controller.js (parse request, call service)
- service.js (rules; may emit events)
- repository.js / API client (Sequelize or provider HTTP call)
- mapper.js (clean DTO)
- controller.js (JSON response)

errors at any step → next(err) → errorHandler → JSON error envelope

3.8 Cross-module communication

- ❑ **Event bus:** billing emits **user.downgraded**; the QuickBooks and Xero services listen and clear that user's connections. Billing never imports the ERP modules.
- ❑ **Shared token logic:** both ERP modules reuse core/oauth/TokenManager through their own client + repository classes.

- **Database aggregator:** modules obtain models from `core/database` rather than from each other.
- **Optional mounting:** `routes/index.js` loads QuickBooks and Xero inside `try / catch`, so removing a module does not break start-up.

4. Excel Add-in Structure (Frontend-React)

4.1 Folder tree

Frontend-React/

```
|— package.json  webpack.config.js

|— manifest.xml      development manifest (localhost:3001)

|— manifest.prod.xml  production manifest

|— assets/           icons (32, 64, 80, 128) and logo

└— src/

    |— taskpane/

    |   |— index.jsx      mounts <App/> after Office.onReady

    |   |— index.html  trialselect.html

    |   |— App.jsx       root: session, views, auth, plans

    |   |— api.js        API_BASE, apiFetch, OAuth, SSE

    |   |— batchDataLoader.js  batched writes, highlighting

    |   |— accountHistory.js  remembers up to 5 previous accounts

    |   |— services/

    |       |— excelService.js    Excel API facade
```

			excelSheetSetup.js	create / style sheets
			excelBatchAppender.js	delta append, clear ranges
			excelDataMappers.js	pure helpers: API payload → rows
			notificationService.js	backend-synced notifications
			taskpane.css	styles.css
			styles/*.css	20 CSS files by area
			components/	React components (section 4.3)
			commands/	commands.html + commands.js (ribbon command stubs)

4.2 Screens (views) managed by App.jsx

View	Component	When shown
loading	Loading indicator	While the saved session is being checked
welcome	Welcome, AccountPicker	Not signed in; choose Google / Microsoft or a remembered account
trial	TrialSelect / TrialSelectModal	Signed in but no plan yet
plans	Plans	Compare and choose Basic / Standard / Pro
payment	Payment	Opens the checkout pop-up for the chosen plan
success	Success screen	After a successful payment
dashboard	Dashboard → ConnectedDashboard	Signed in with an active plan

4.3 Components (src/components)

Component	Purpose
Welcome.jsx, AccountPicker.jsx	Sign-in screen and previous-account chooser
TrialSelect.jsx, TrialSelectModal.jsx	Choose free trial or paid plan
Plans.jsx, Payment.jsx	Plan comparison and checkout hand-off
Dashboard.jsx	Main container: not-connected vs connected state, wires the hooks below
ConnectedDashboard.jsx	Tabs and panels once a company is connected
AccountMenu.jsx	User menu: plan, sign-out, switch account
NotificationDrawer.jsx	Notification list with badge and clear actions
TrialExpiredModal.jsx	Upgrade prompt when the trial ends
GlobalErrorBanner.jsx	Offline / backend-down / session-expired banners
ToastContainer.jsx	Short-lived messages
ErrorBoundary.jsx	Catches render errors for the whole task pane
ui.js	Shared UI helpers

components/dashboard

File	Purpose
useCompanyConnections.js	Hook: list, connect, reconnect, activate, rename, disconnect companies; limits and counts
useMasterDataSync.js	Hook: Setup sheets, Pull Master Data and Refresh Schedule (streaming, cursor, notifications, trial guards)
CompanyListSection.jsx	List of connected companies with status
CompanyContextMenu.jsx	Right-click menu per company
ChangeCompanyModal.jsx, RenameCompanyModal.jsx	Switch and rename dialogs
SyncActionsPanel.jsx	Buttons for Setup, Pull and Refresh
SubscriptionCard.jsx	Current plan and trial countdown
JournalSelector.jsx	Journal selection control
DashboardIcons.jsx	SVG icon set

4.4 Excel workbook layout produced by the add-in

Sheet	Content
1.Master_Data	Five colour-coded blocks: Company, Accounts, Classes, Locations and Entities (customers / vendors). Header styling is created by excelSheetSetup.js
2.Input	Input sheet scaffolded at setup for accrual entries

Master data is written in batches (batchDataLoader.js). On later refreshes only new or updated records are appended below the existing rows and highlighted in grey, with de-duplication by ID.

4.5 Build outputs

Webpack produces taskpane.html, commands.html and trialselect.html, copies assets/, the CSS files and both manifests, and serves them at <https://localhost:3001> in development. API_BASE is injected at build time.

5. Admin Dashboard Structure (Admin)

Admin/

├── package.json vite.config.ts tsconfig*.json eslint.config.js index.html

├── public/ favicon.svg, icons.svg

└── src/

 ├── main.tsx App.tsx entry; <AppProvider> + <Toaster> + <AppRoutes>

 ├── routes/AppRoutes.tsx login or routed layout

 ├── context/AppContext.tsx global state + sample data

 ├── layouts/AdminLayout.tsx Sidebar + Header + <Outlet/>

 ├── components/layout/ Sidebar.tsx, Header.tsx

 ├── modules/ one folder per page (below)

Route	Module	Purpose
(not signed in)	auth/Login.tsx	Sign-in / sign-up form and Google sign-in pop-up; stores token and user
/	dashboard/Dashboard.tsx	Summary statistics
/clients	clients/Clients.tsx	Client list
/master-data	master-data/MasterData.tsx	Master accounts data
/bulk-upload	bulk-upload/BulkUpload.tsx	Upload workflow and history
/input	input/Input.tsx	Manual input form
/schedules	schedules/Schedules.tsx	Accrual schedules
/journal-entries	journal-entries/JournalEntries.tsx	Journal entries
/exports	exports/Exports.tsx	Export history
/workpapers	workpapers/Workpapers.tsx	Workpapers
(not routed)	invoice/Invoice.tsx, class-data/ClassData.tsx	Present in the source but not registered in AppRoutes.tsx

Note The dashboard pages currently read and update sample data held in `AppContext` (clients, schedules, accounts, journal entries, exports, workpapers, uploads). Only the sign-in screen calls the backend, so these pages are not yet connected to the database.

6. Tests

Location	Type	Covers
tests/unit/core/	Unit	helpers.test.js, oauth.test.js (token manager behaviour)
tests/unit/modules/admin quickbooks xero/	Unit	Mapper output (DTOs)
tests/unit/modules/excelValidation/	Unit	parser, schemaValidator, dataTypeValidator, diffEngine, apiComparator, dbComparator, reportGenerator

tests/integration/	Integration (Supertest)	Route behaviour for admin, QuickBooks, Xero and excel-validation with services mocked
scripts/loadtest.js	Load	Autocannon throughput and latency on representative endpoints

Run with npm test in Backend/Backend. The front-end projects have no automated test suites.

7. Dependencies by Project

Backend (production)

Package	Use
express 5, cors, helmet, express-session	Web server, CORS, security headers, sessions
sequelize, mysql2 (pg, pg-hstore, sqlite3 also installed)	ORM and MySQL driver
jsonwebtoken, bcrypt	JWT and password hashing
joi, express-rate-limit	Validation and throttling
axios	Calls to QuickBooks, Xero, Google, Microsoft
exceljs	Read and write .xlsx files
pino, pino-pretty, dotenv	Logging and environment loading

Dev tools: jest, supertest, nodemon, concurrently, cross-env, eslint, prettier, autocannon.

Front ends

Project	Main packages
Frontend-React	react 19, react-dom, core-js, regenerator-runtime; dev: webpack 5, babel, office-addin-debugging, office-addin-dev-certs
Admin	react 19, react-router-dom 7, @reduxjs/toolkit, react-redux, @tanstack/react-table, react-hook-form, zod, axios, sonner, lucide-react; dev: vite, typescript, eslint

8. Conventions and How to Extend

8.1 Naming and style

- ☐ Backend files use CommonJS (require), classes with static methods for services, and the module file names listed in section 3.4.
- ☐ Database columns are snake_case (except notifications, which is camelCase); API responses and DTOs use camelCase.
- ☐ Errors are thrown as `AppError` subclasses and passed to `next(err)`; controllers do not build their own error responses.
- ☐ Browser storage keys in the add-in are prefixed `fa_`.

8.2 Adding a new backend module

1. Create `src/modules/<name>/` with `model.js`, `repository.js`, `service.js`, `controller.js`, `routes.js`, `index.js` (and `mapper.js` if outside data is transformed).
2. Register the model and associations in `core/database/index.js`.
3. Add Joi schemas to `core/validation/schemas.js` and use `validate(...)` in the routes.
4. Protect routes with `authenticate` (and `authorize(...)` if role-restricted); filter every query by `req.user.userId`.
5. Mount the router in `src/routes/index.js`.
6. Add unit tests for the mapper / service and a Supertest integration test for the routes.

8.3 Adding another ERP provider

1. Copy the QuickBooks or Xero module structure.
2. Implement `IOAuthClient` and `IOAuthTokenRepository`, and create a `TokenManager` instance for the provider.
3. Add provider constants to `core/constants` and settings to `core/config`.
4. Add the provider to the shared `/api/connections` and `/api/pull-master-data` handlers and to the Joi platform enum.
5. Add the provider's domains to both Office manifests.

9. Files to Review or Clean Up

File / area	Observation
core/redis.js	Not imported anywhere; needs config.REDIS and the redis package, neither of which exists
__db_stub__.js, debug_output.html	Test / debug leftovers; confirm and remove from the delivered package
config/logger.js and core/logger/index.js	Two logger implementations (pino vs console). Choose one
localhost.session.sql, root package-lock.json	Empty editor session file and a lock file without a root package.json
Backend/package-lock.json	Lock file one level above the actual project folder Backend/Backend
Docs (README.md, BACKEND_DOCUMENTATION.md)	Mention SQLite, a 15-minute token and a fin-12.30/ folder name; the code uses MySQL, a 1-hour token and the folder is ERP-main
Admin/src/modules/invoice, class-data	Pages not routed
Unused packages	pg, pg-hstore, sqlite3 (backend) and office-js (Admin) are installed but not used
Admin sidebar	Calls /api/launch-excel, which has no backend route